

Development Plan: Llama.cpp GUI Wrapper for Fedora

1. Introduction

This document outlines the development plan for a Fedora desktop application that provides a user-friendly Graphical User Interface (GUI) wrapper for llama.cpp . The primary goal is to enable users to easily select and configure llama.cpp models and settings without requiring direct interaction with the command-line interface (CLI).

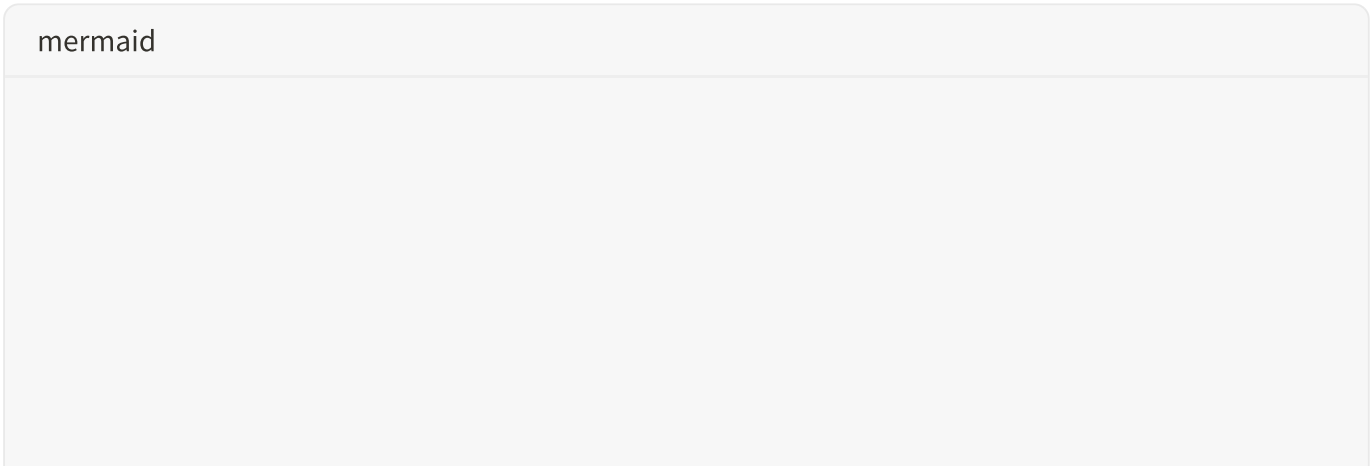
2. Application Architecture

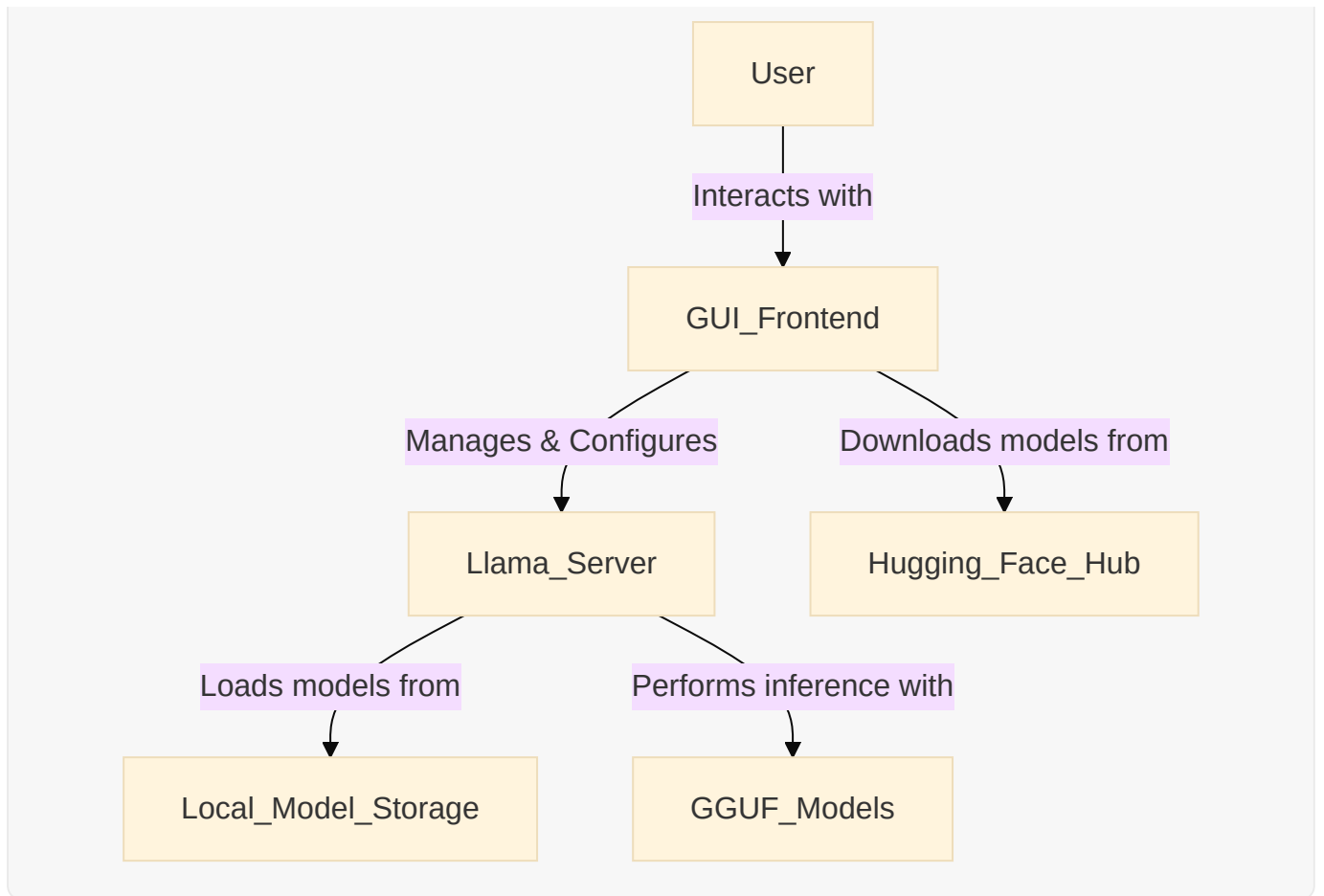
Given the rapid development pace of llama.cpp and its frequent updates, the application architecture will prioritize modularity and clear separation of concerns. This approach aims to minimize the impact of llama.cpp updates on the GUI and facilitate seamless integration of new llama.cpp versions.

The application will follow a client-server architecture, where the GUI acts as a client communicating with a locally running llama-server instance. This approach leverages the existing robust API provided by llama.cpp 's server component, ensuring stability and compatibility with future llama.cpp updates.

2.1. Components

- **GUI Frontend:** Developed using Python and GTK4, providing the user interface for interaction.
- **Llama.cpp Server (llama-server):** The core backend for model inference, exposed via a REST API. The GUI will manage the lifecycle of this server (start, stop, configuration).
- **Model Storage:** GGUF model files will be stored locally on the user's system in a designated directory.





3. Feature Set

The application will include the following core features:

- **Server Management:**
 - Provide an in-app mechanism to check for and download the latest `llama.cpp` binaries or source code, and rebuild `llama-server` if necessary.
 - Manage the `llama-server` process lifecycle (start, stop, restart).
 - Display server status (running/stopped, port, `llama.cpp` version).
- **Model Management:**
 - Browse local GGUF model files.
 - Select a model to load into the `llama-server`.
 - Download GGUF models directly from Hugging Face Hub [1].
 - Delete local model files.
- **Configuration Settings:**
 - Adjust `llama-server` parameters (e.g., context size, GPU layers, thread count, temperature, `top_k`, `top_p`, `repeat_penalty`).

- Save and load configuration profiles for different models or use cases.
- **Inference Interface:**
 - Text input area for user prompts.
 - Display area for `llama.cpp`'s generated responses.
 - Clear conversation history.
- **User Experience:**
 - Intuitive and responsive GUI.
 - Error handling and informative messages for server issues or invalid configurations.

4. Technical Stack

4.0. Adaptability to Llama.cpp Updates

To accommodate the rapid update cycle of `llama.cpp`, the application will implement the following strategies:

- **Dynamic Linking/Separate Installation:** Explore options for dynamically linking against `llama.cpp` libraries or managing `llama.cpp` as a separate, user-updatable component rather than bundling a static version within the RPM. This will allow users to update `llama.cpp` independently of the GUI application.
- **API-Centric Design:** Rely heavily on the `llama-server` REST API for all interactions. This decouples the GUI from `llama.cpp`'s internal implementation details, making the GUI more resilient to changes in `llama.cpp`'s core.
- **Configuration Abstraction:** Abstract `llama-server` configuration parameters within the GUI to handle potential changes in parameter names or structures between `llama.cpp` versions. The GUI will dynamically adapt to available parameters or provide mappings.
- **Version Compatibility Checks:** Implement checks to ensure compatibility between the GUI and the installed `llama.cpp` version, guiding users to update if necessary.

4.1. GUI Framework

- **Language:** Python
- **Toolkit:** [PyQt/PySide with QtQuick and Kirigami](#) [2]
- **Styling:** Kirigami (for native KDE Plasma look and feel, and adherence to KDE Human Interface Guidelines).

Python with PyQt/PySide and Kirigami offers a native and well-integrated experience on Fedora KDE Plasma, aligning with the KDE desktop environment. This choice ensures the

application will have a consistent and modern appearance within KDE Plasma, leveraging the powerful Qt framework and KDE's design language.

4.2. Llama.cpp Interaction

- **Backend:** `llama-server` (part of the `llama.cpp` project).
- **Communication:** REST API calls to the `llama-server` (e.g., `/completion` , `/models` , `/props`). The `llama-server` provides an OpenAI-compatible API, simplifying interaction [3].

4.3. Model Downloading

- **Library:** `huggingface_hub` Python library [1]. This library provides programmatic access to download models from the Hugging Face Hub, which is a common source for GGUF models.

4.4. Packaging

- **Format:** RPM package for Fedora Linux.
- **Guidelines:** Adherence to [Fedora Packaging Guidelines](#) [4], specifically for Python applications. This includes proper dependency management, file system hierarchy, and bundling policies.

5. References

1. [Hugging Face Hub Documentation](#) - Information on downloading models programmatically.
2. [KDE Developer - A full Python + Kirigami application](#) - Official tutorial for Python and Kirigami application development.
3. [llama-cpp-python server documentation](#) - Details on the OpenAI-compatible API provided by `llama-server` .
4. [Fedora Python Packaging Guidelines](#) - Official guidelines for packaging Python applications for Fedora.